

HDOC Day-16

NAME: Yug Nirwal

SAP ID: 590033354

SUBJECT: C programming

DATE: 9-09-26

Question 1 - Check Digit Order

Write a menu-driven C program to check whether the digits of a positive integer are in increasing or decreasing order from left to right.

Required approach: menu-driven program, switch-case, separate void user-defined functions, and simple function calls without passing arguments.

1. Algorithm

1. Start the program.
2. Display a menu: 1 for increasing order, 2 for decreasing order, and 3 to exit.
3. Read the user's choice in main() and use switch-case.
4. For option 1, call `increasingOrder()`. The function reads the positive integer, reverses it, and compares adjacent digits from left to right. If every next digit is greater than the previous digit, display that the digits are in increasing order.
5. For option 2, call `decreasingOrder()`. The function reads the positive integer, reverses it, and compares adjacent digits. If every next digit is smaller than the previous digit, display that the digits are in decreasing order.
6. For an invalid menu choice, display an appropriate message.
7. Stop.

2. Test Case Table

Test Case	Menu Choice	Input Number	Expected Result
1	1	123579	Digits are in increasing order
2	1	124431	Digits are not in increasing order
3	2	975310	Digits are in decreasing order
4	2	986679	Digits are not in decreasing order

3. C Program

```
#include <stdio.h>

void increasingOrder(void);
```

```

void decreasingOrder(void);

int main(void)
{
    int choice;

    printf("1. Check increasing order\n");
    printf("2. Check decreasing order\n");
    printf("3. Exit\n");
    printf("Enter your choice: ");
    scanf("%d", &choice);

    switch (choice)
    {
        case 1:
            increasingOrder();
            break;
        case 2:
            decreasingOrder();
            break;
        case 3:
            printf("Program ended.\n");
            break;
        default:
            printf("Invalid choice.\n");
    }
    return 0;
}

void increasingOrder(void)
{
    long long n, temp, rev = 0;
    int previous, current, increasing = 1;

    printf("Enter a positive integer: ");
    scanf("%lld", &n);

    temp = n;
    while (temp > 0)
    {
        rev = rev * 10 + temp % 10;
        temp = temp / 10;
    }

    previous = rev % 10;
    rev = rev / 10;
    while (rev > 0)
    {
        current = rev % 10;
        if (current <= previous)
        {
            increasing = 0;
            break;
        }
        previous = current;
        rev = rev / 10;
    }

    if (increasing)
        printf("Digits are in increasing order.\n");
    else
        printf("Digits are not in increasing order.\n");
}

void decreasingOrder(void)
{
    long long n, temp, rev = 0;
    int previous, current, decreasing = 1;

    printf("Enter a positive integer: ");
    scanf("%lld", &n);

```

```

temp = n;
while (temp > 0)
{
    rev = rev * 10 + temp % 10;
    temp = temp / 10;
}

previous = rev % 10;
rev = rev / 10;
while (rev > 0)
{
    current = rev % 10;
    if (current >= previous)
    {
        decreasing = 0;
        break;
    }
    previous = current;
    rev = rev / 10;
}

if (decreasing)
    printf("Digits are in decreasing order.\n");
else
    printf("Digits are not in decreasing order.\n");
}

```

4. Compile, Execute and Test

Save the program as q1.c. In the terminal, compile it with:

```

gcc q1.c -o q1
./q1

```

Test	Actual Output	Status
1 - choice 1, input 123579	Digits are in increasing order.	PASS
2 - choice 1, input 124431	Digits are not in increasing order.	PASS
3 - choice 2, input 975310	Digits are in decreasing order.	PASS
4 - choice 2, input 986679	Digits are not in decreasing order.	PASS

```
(kali@kali) - [~/Desktop/cprog]
$ ./q1
1. Check increasing order
2. Check decreasing order
3. Exit
Enter your choice: 1
Enter a positive integer: 123579
Digits are in increasing order.
```

```
(kali@kali) - [~/Desktop/cprog]
$ ./q1
1. Check increasing order
2. Check decreasing order
3. Exit
Enter your choice: 1
Enter a positive integer: 124431
Digits are not in increasing order.
```

```
(kali@kali) - [~/Desktop/cprog]
$ ./q1
1. Check increasing order
2. Check decreasing order
3. Exit
Enter your choice: 2
Enter a positive integer: 975310
Digits are in decreasing order.
```

```
(kali@kali) - [~/Desktop/cprog]
$ ./q1
1. Check increasing order
2. Check decreasing order
3. Exit
Enter your choice: 2
Enter a positive integer: 986679
Digits are not in decreasing order.
```

Result: The program correctly checks increasing and decreasing digit order using separate void functions and switch-case.

Question 2 - Find Largest/Smallest and Second Largest/Smallest Digits

Write a separate menu-driven C program to find the largest and smallest digit, or the second largest and second smallest distinct digit, in a positive integer.

Required approach: menu-driven program, switch-case, separate void user-defined functions, and simple function calls without passing arguments.

1. Algorithm

1. Start the program.
2. Display a menu: 1 for largest and smallest digit, 2 for second largest and second smallest distinct digit, and 3 to exit.
3. Read the choice in main() and use switch-case.
4. For option 1, call largestSmallest(). Read the number, examine every digit, and update the largest and smallest values.
5. For option 2, call secondLargestSmallest(). Read the number and mark which digits from 0 to 9 occur. Scan upward to obtain the smallest and second smallest distinct digits, and scan downward to obtain the largest and second largest distinct digits.
6. If fewer than two distinct digits exist, display that second largest/smallest cannot be found.
7. For an invalid menu choice, display an appropriate message.
8. Stop.

2. Test Case Table

Test Case	Menu Choice	Input Number	Expected Result
1	1	583204	Largest = 8, Smallest = 0
2	1	99991	Largest = 9, Smallest = 1
3	2	583204	Second largest = 5, Second smallest = 2
4	2	7777	Second largest/smallest cannot be found

3. C Program

```
#include <stdio.h>

void largestSmallest(void);
void secondLargestSmallest(void);

int main(void)
{
    int choice;
```

```

printf("1. Find largest and smallest digit\n");
printf("2. Find second largest and second smallest digit\n");
printf("3. Exit\n");
printf("Enter your choice: ");
scanf("%d", &choice);

switch (choice)
{
    case 1:
        largestSmallest();
        break;
    case 2:
        secondLargestSmallest();
        break;
    case 3:
        printf("Program ended.\n");
        break;
    default:
        printf("Invalid choice.\n");
}
return 0;
}

void largestSmallest(void)
{
    long long n;
    int digit, largest = 0, smallest = 9;

    printf("Enter a positive integer: ");
    scanf("%lld", &n);

    while (n > 0)
    {
        digit = n % 10;
        if (digit > largest)
            largest = digit;
        if (digit < smallest)
            smallest = digit;
        n = n / 10;
    }

    printf("Largest digit = %d\n", largest);
    printf("Smallest digit = %d\n", smallest);
}

void secondLargestSmallest(void)
{
    long long n;
    int digit, present[10] = {0};
    int i, count = 0;
    int smallest = -1, secondSmallest = -1;
    int largest = -1, secondLargest = -1;

    printf("Enter a positive integer: ");
    scanf("%lld", &n);

    while (n > 0)
    {
        digit = n % 10;
        present[digit] = 1;
        n = n / 10;
    }

    for (i = 0; i <= 9; i++)
    {
        if (present[i])
        {
            count++;
            if (smallest == -1)
                smallest = i;

```

```

        else if (secondSmallest == -1)
            secondSmallest = i;
    }
}

for (i = 9; i >= 0; i--)
{
    if (present[i])
    {
        if (largest == -1)
            largest = i;
        else if (secondLargest == -1)
            secondLargest = i;
    }
}

if (count < 2)
    printf("Second largest/smallest cannot be found.\n");
else
{
    printf("Second largest digit = %d\n", secondLargest);
    printf("Second smallest digit = %d\n", secondSmallest);
}
}

```

4. Compile, Execute and Test

Save the program as q2.c. In the terminal, compile it with:

```

gcc q2.c -o q2
./q2

```

Test	Actual Output	Status
1 - choice 1, input 583204	Largest digit = 8; Smallest digit = 0	PASS
2 - choice 1, input 99991	Largest digit = 9; Smallest digit = 1	PASS
3 - choice 2, input 583204	Second largest digit = 5; Second smallest digit = 2	PASS
4 - choice 2, input 7777	Second largest/smallest cannot be found.	PASS

Terminal Test Case Screenshots

```

(kali@kali) - [~/Desktop/cprog]
$ ./q2
1. Find largest and smallest digit
2. Find second largest and second smallest digit
3. Exit
Enter your choice: 1
Enter a positive integer: 583204
Largest digit = 8
Smallest digit = 0

```

```
(kali@kali) - [~/Desktop/cprog]
$ ./q2
1. Find largest and smallest digit
2. Find second largest and second smallest digit
3. Exit
Enter your choice: 1
Enter a positive integer: 99991
Largest digit = 9
Smallest digit = 1
```

```
(kali@kali) - [~/Desktop/cprog]
$ ./q2
1. Find largest and smallest digit
2. Find second largest and second smallest digit
3. Exit
Enter your choice: 2
Enter a positive integer: 583204
Second largest digit = 5
Second smallest digit = 2
```

```
(kali@kali) - [~/Desktop/cprog]
$ ./q2
1. Find largest and smallest digit
2. Find second largest and second smallest digit
3. Exit
Enter your choice: 2
Enter a positive integer: 7777
Second largest/smallest cannot be found.
```

Result: The program correctly finds the required digit values using separate void functions, simple function calls, and switch-case.